



Starter Kit

Setup walkthrough — wiring the real files into a running agent, start to finish

The actual scripts this project runs on, sanitized and generalized, plus this copy-paste walkthrough for standing them up on your own box.

beaconwake.com

Contents

- 1. A small VM
- 2. A non-root sudo user
- 3. Lock down SSH
- 4. Node, and Claude Code
- 5. This kit
- 6. A Telegram bot
- 7. Wire the path
- 8. Cron
- 9. Test it end to end, manually, before trusting cron
- 10. Day-one hardening

1. A small VM

Any cheap Linux VM works (1 vCPU / 1–2GB RAM is plenty for this). Provision one from any provider, note its public IP.

2. A non-root sudo user

Don't run this as root day to day.

```
adduser youruser
usermod -aG sudo youruser
```

Give it passwordless sudo if you want the agent able to install packages or edit system config unattended — put this in its own file under `/etc/sudoers.d/` (not appended to the end of `/etc/sudoers` directly; `sudoers.d` is processed after the main file, so a rule there reliably wins over the default `%sudo` group line, which otherwise can silently override an equivalent rule placed earlier in `/etc/sudoers` itself):

```
echo "youruser ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/youruser
sudo chmod 0440 /etc/sudoers.d/youruser
```

3. Lock down SSH

Key-only login, no root over SSH if you can avoid it:

```
ssh-copy-id youruser@your-vm-ip
```

Then in `/etc/ssh/sshd_config`: `PasswordAuthentication no`, and `sudo systemctl restart ssh`. Verify you can still log in with the key in a *second* terminal before closing the first.

4. Node, and Claude Code

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install --lts
npm install -g @anthropic-ai/claude-code
claude --version
```

5. This kit

```
git clone <wherever you're keeping this> agent
cd agent
mv AGENT.md.template AGENT.md      # edit YOUR_NAME
mv NOTES.md.template NOTES.md
mv ASK.md.template ASK.md
mkdir -p keys logs
chmod +x wake.sh notify.sh check_replies.sh
```

`AGENT.md` is the one file that matters most — it's read at the start of every waking. Edit `YOUR_NAME` and adjust the rules to taste; the ones in the template (nothing illegal, never claim to be human, keys stay out of git, irreversible/gray actions go to ASK.md and wait) are a reasonable floor, not a ceiling.

6. A Telegram bot

Message `@BotFather` on Telegram, `/newbot`, follow the prompts — you get a bot token. Then message your new bot anything at all (it needs a first message from you to have a chat to reply into), and hit:

```
https://api.telegram.org/bot<TOKEN>/getUpdates
```

Your chat id is in the JSON response (`message.chat.id`). Put both in `keys/telegram.env` :

```
TELEGRAM_BOT_TOKEN=xxxx
TELEGRAM_CHAT_ID=xxxx
```

```
chmod 600 keys/telegram.env
echo "keys/" >> .gitignore
```

Test it: `./notify.sh "hello from the new box"` should land in your Telegram within a couple seconds.

7. Wire the path

Edit `wake.sh`'s `PROJECT_DIR` line to the absolute path where you cloned this (e.g. `/home/youruser/agent`). If you want the news/weather digest, also fill in `digest.sh`'s gridpoint URL and contact email (see the comments in that file), and wire it into `wake.sh` if you want it sent every wake, or gate it to once a day the way this kit's source project does (check the local hour, no-op unless it matches, with a small state file as a backstop against double-sends within that hour).

8. Cron

```
crontab -e
```

Add a line like (5x/day, ~5h apart, UTC):

```
0 0,5,10,14,19 * * * /home/youruser/agent/wake.sh
```

9. Test it end to end, manually, before trusting cron

```
./wake.sh
```

Watch `logs/` for the new log file, and confirm a Telegram message arrives. Only once this works standalone should you trust the cron timer to fire it unattended — debugging a cron-only failure with no console access is much harder than catching it here first.

10. Day-one hardening

Not required to get running, but cheap and worth doing immediately on a box that's going to sit on the open internet unattended:

```
sudo apt-get install -y fail2ban unattended-upgrades
sudo ufw allow OpenSSH
sudo ufw enable
```

That's the whole loop: cron fires `wake.sh`, which hands the agent `AGENT.md` plus its own history (`NOTES.md` / `ASK.md` / `memory/`), the agent does something and writes down what and why, `notify.sh` tells you, and the next waking picks up wherever this one left off.